

Fundamentals of data structures

LEARNING OBJECTIVES

After studying this chapter, learners will be able to:

- Know what data structures are
- Know the basic operations performed in a data type
- Know the difference between primitive and non-primitive data structures
- Know some well-known data structures
- Understand abstract data types

We will discuss the basics of data structure and abstract data types. Every computer program has two parts: an algorithm and a data structure. The algorithm tells us what the program will do and operates on variables, whereas the data structure is a group of memory locations used to retain the information used by the algorithm.

2.1 INTRODUCTION

Only the primitive data that are defined can be manipulated by a digital computer. The user does not need to apply any additional effort while manipulating the primitive data. However, in our real-world applications, we use a variety of data types other than primitive data. Manipulation of these data necessitates the completion of the following tasks.

1. **User data storage representation:** User data should be stored in a format that a computer can comprehend.
2. **Data retrieval:** Data saved on a computer should be retrieved in a way that the user can comprehend.
3. **User data transformation:** Various actions must be conducted on user data to translate from one form to another.

People who create and develop computer programs such as system software or application software must have a basic understanding of data structures. We know that data is represented by data values that are either temporarily stored in a program's data area or permanently recorded on a file. Different data values are frequently related to one another. These data values must be arranged for programs to create a connection. The term *data structure* refers to a structured collection of data. To access and process structured data, the programs must follow a set of rules.

Data structure = Organization of data * Operations on the data

Data structure refers to the logical or mathematical paradigm of a data organization. Data can be structured in many different ways. The decision to adopt a data model is influenced by two factors. Firstly, it must have a rich structure that mimics the true relationship of the facts in the real world. Secondly, the structure should be simple enough that the data can be quickly processed as needed.

2.2 CLASSIFICATION OF DATA STRUCTURES

Data structures are generally divided into two broad categories:

1. Primitive data structures
2. Non-primitive data structures

Machine instructions work directly on primitive data structures. These data structures have different representations on different machines. The category includes integers, floating-point numbers, characters, strings, lists, tuples, sets, and dictionaries in Python.

The complexity of non-primitive data structures is higher than that of primitive data structures. The non-primitive data structures are made from primitive data structures. Non-primitive data structures are focused on the organized collection of homogeneous or heterogeneous data items.

The operations performed on the data structure must be considered while designing an effective data structure.

Further, the non-primitive data structure is categorized into linear data structures, non-linear data structures, and file structures.

The data elements in linear data structures are ordered in a linear sequence. In non-linear data structures, the data items arranged are not in a sequence (Figure 2.1).

The most commonly used operations in data structures are broadly categorized into seven types:

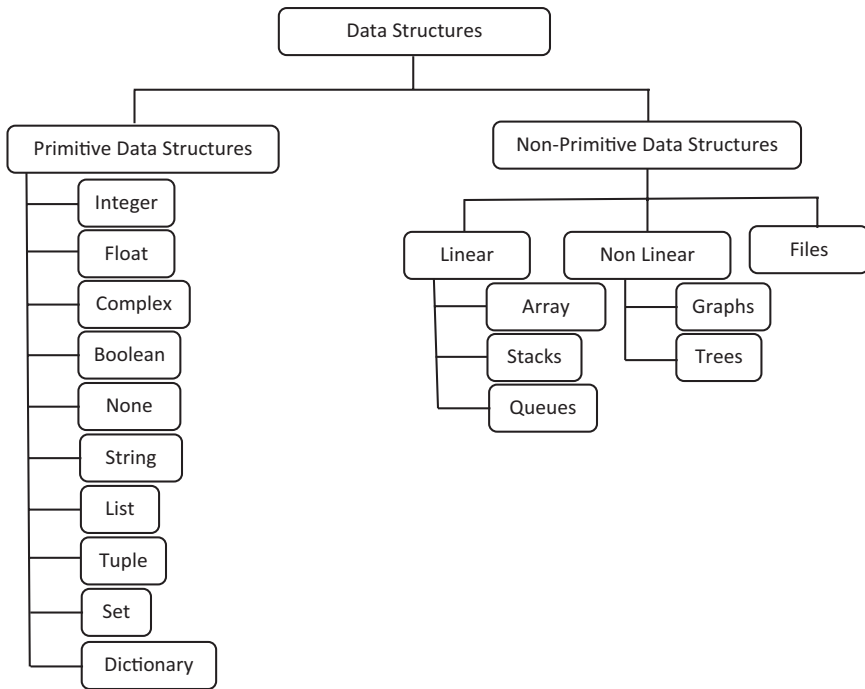


Figure 2.1 Data structure classification

- i. Create
- ii. Delete
- iii. Select
- iv. Update
- v. Search
- vi. Sort
- vii. Merge

The **Create** operation is used to reserve memory for program items. Declarative statements can be used to reserve the memory that can be done at compile time or runtime, depending on the programming language.

The **Delete** action releases memory reserved for a data item in the specified data structure.

The **Selection** operation is used to get a specific data item from a data structure.

The **Update** operation is used to make changes to a data item.

The **Search** procedure determines whether or not the targeted data item exists. It may also locate all the elements that satisfy a set of criteria.

Sorting involves putting data items in ascending or descending order.

Merging refers to the process of combining data items of two different sorted lists into a single list.

2.3 DESCRIPTIONS OF VARIOUS DATA STRUCTURES

Data structures can be classified into two categories: primitive and non-primitive.

2.3.1 Primitive data structures

Various primitive data structures come with the programming language. The primitive data structures that come with Python are discussed here.

2.3.1.1 Integer

Numeric data types include integers, floats, complex numbers, and Booleans. Data in some variables are kept as whole numbers, for instance, the number of students in a class. An integer-type variable is used to save the value of such a sort of query. Integers are the only data types that do not support decimal places. In other words, only the complete number is required. It includes the values positive, negative, and zero. Integers such as 8, -432, and 0 are examples. Take a look at the expression $x = 3 + 5$. Python computes the right side of the equal sign, stores the result in memory, and connects the memory location with the identifier x . In other words, it assigns that location a name, and because the result of the right-hand expression is an integer, the type of this variable will be “int.” We have a variable in this scenario, and the identifier for this variable is x . A variable’s name can alternatively be referred to as x . The variable has an integer value of 8 and is of the type int. The statement `print(type(x))` returns a *class int*, indicating that its type is an integer, which can be confirmed.

2.3.1.2 Float

Some variables have fractional values. When asked, “How tall are you?” we normally respond with 5.6 feet. We’ll need a floating-point representation to hold the height value. Floating point numbers are those that have a decimal point, for example, 43.9, -234.1, and 00.4. Take the statement $y = 3 / 2$ as an example. Python calculates the right side of the equal sign, saves the result, and associates it with the identifier y in memory. Because the result is a floating-point number, this variable’s type will be float. The value of the variable is 1.5. This can be verified by “`print(type(y))`.”

2.3.1.3 Complex

One-dimensional numbers are integers and floats. We also require two-dimensional integers on occasion. Another numeric data format, complex number, is used to represent a two-dimensional number. The ordered pair $x+yj$ is used to represent complex numbers, where x and y are numeric data. The real part is x , while the imaginary part is y . Complex numbers include $9+8j$ and complex $(9, 8)$. The most common operations performed on complex data types are addition, subtraction, multiplication, and division.

2.3.1.4 Boolean

Boolean is a binary variable that can have one of two potential values: 0 (False or F) or 1 (True or T). For instance, if the inquiry is, “Are you happy?” The response is either yes or no. It means you’ll need a variable to hold data of the Boolean type. The operations used with the Boolean data type were AND, OR, and XOR.

Let's have a look at the bool type. Consider the proposition $z = \text{False}$. True or False are both keywords in Python, and the first letter is uppercase. Make sure that they are not typed as true or false (in lowercase letters).

Python computes the right side of the equal sign, stores the result in memory, correlates that memory address with the identifier z , and since the right-side result is a Boolean, the type is bool.

It's also worth noting that z was previously specified as “hello” and had the typed string.

However, with the sentence $z = \text{False}$, Python immediately assigns the value False to z . So, the prior value of z , “hello,” is no longer valid, and its new type is bool. In Python, this is done automatically, so we don't have to worry about it.

Python includes several other built-in data types, including string, list, tuple, dictionary, and set. We also have data structures like arrays that must be imported before they can be used.

2.3.1.5 None

This data type stands for “no value” or “null.” It has no meaning when it comes to “0,” “empty,” “False,” or “undefined.” It is a true data type; it can be assigned to variables and takes up memory like any other data type. When a function does not include a return statement and we want to print the value returned by the function, we use the “None” data type.

2.3.1.6 String

If you're asked, “What's your name?” the string type of variable is used to answer this question. It's simply a string of letters, numbers, and special

characters. Immutability is a property of the string data type. Concatenation, identifying substrings, and many other operations are performed on string-type data.

Now consider the statement `z = "hi."` Python calculates the right side once more, saves the result to memory, and connects that memory location with the identifier `z`. Because the right side contains a collection of characters, this variable's type is `str`, which stands for string. Python now has a variable with the name `z`, the value `"hi."`, and the type `str`.

2.3.1.7 List

This sort of data is used to store numerous pieces of information at the same time under a single variable name. It is analogous to an array in a programming language such as C. C-array, on the other hand, is a homogeneous data type, whereas a list does not have to be of the same data type. The data in a list is enclosed by a pair of square braces `[ ]`.

2.3.1.8 Tuple

The main difference between a tuple and a list is that a tuple is immutable. It is made up of unchanging and indexed data. The data in a tuple is enclosed by a pair of round brackets `( )`.

2.3.1.9 Set

It's a collection of datasets that aren't in any particular sequence and aren't indexed. It's an immutable data type that stores one-of-a-kind information (no duplicates). A pair of curly braces `{ }` surrounds the data in a set.

2.3.1.10 Dictionary

A dictionary is a specific data type in which data is stored in pairs of (key:value). It's an unsorted, changing, and indexed collection. Dictionary works similarly to a real-world dictionary. A dictionary's keys must be unique and immutable data types like strings, integers, and tuples, but the key-values can be repeated and of any kind.

2.3.2 Non-primitive data structures

Non-primitive data structures, also known as composite or derived data structures, are data structures that are not built-in or predefined in a programming language. Unlike primitive data types that hold a single value, non-primitive data structures can hold multiple values and are composed of primitive data types or other non-primitive data structures. The

non-primitive data structures provide flexibility and efficiency in storing and manipulating complex data in programming languages.

2.3.2.1 Linear data structures

Linear data types are those data types that are ordered by nature. In other words, the elements are accessed one after another in a sequence.

2.3.2.2 Array

This data type holds multiple data of homogeneous data types together and is accessed by an index. In Python, an array can be implemented using primitive data types such as a list or a dictionary. It can also be implemented using a non-primitive data type by importing separate packages like NumPy and array. The elements in an array are accessed sequentially. We can implement an array with the help of a list or a dictionary.

2.3.2.3 Stack

A stack is also a linear data type, but it has a special way of inserting and deleting data. In a stack, the insertion and deletion take place at one end called the *top of the stack*. As a result, the element that is inserted last is to be deleted first so it is also called *last in, first out* (LIFO). For example, as shown in Figure 2.2, we can think of a stack of books on a table. We can remove a book from the top, and also we can put a book on the top of the stack. The operation of inserting and deleting is also known as *push* and *pop*, respectively. Another operation in the stack is *peek*, which returns the element at the top of the stack.

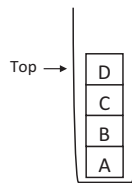


Figure 2.2 Stack

The concept of stack can be implemented using a list, dictionary, class, or queue module.

2.3.2.4 Queue

A queue is also a linear data type, but it has a special way of inserting and deleting data. In a queue, insertion occurs at one end, known as the *rear*,

while deletion occurs at the opposite end, known as the *front*. As a result, the element that is placed first is also the first to be deleted, hence the term *first in, first out* (FIFO). For example, we can think of people standing in a row waiting for a bus. The person who arrives first becomes the first person in the queue and they get onto the bus first (Figure 2.3).



Figure 2.3 Queue

The concept of queue can be implemented using a list, dictionary, class, or queue module.

2.3.3 Non-linear data structures

Non-linear data types are those data types that are not in a specific order. In other words, the elements are not accessed one after another in a sequence.

2.3.3.1 Tree

This is a non-linear data structure in which there is no unique predecessor and successor. This data structure represents a hierarchical relationship between the elements. It is a collection element that is identified by a unique element called the root and the sub-trees are children of the root (Figure 2.4).

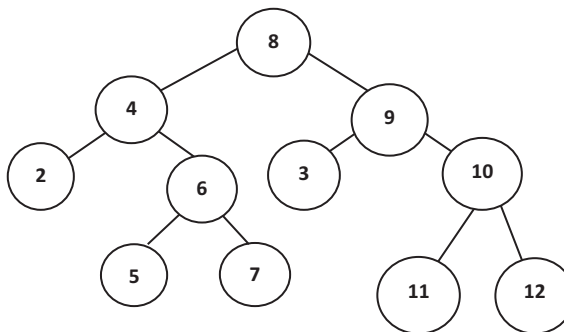


Figure 2.4 Tree

The concept of tree can be implemented using a list or a class.

2.3.3.2 Graph

It is a mathematical non-linear data structure capable of representing any kind of structure. A graph is made up of nodes (also known as vertices) that are linked together by edges (also called arcs) (Figure 2.5).

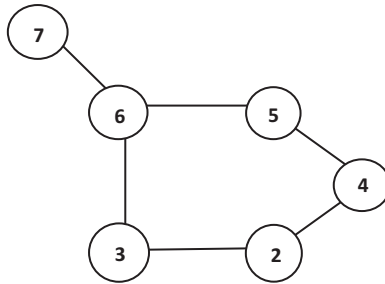


Figure 2.5 Graph

Graphs contain some key terms:

- When two nodes are joined by an edge, they are referred to as neighbors.
- The degree of a node is the number of other nodes to which it is connected (i.e., the number of neighbors that it has).
- A loop is an edge that connects one node to itself.
- A path is a series of nodes linked together by edges.
- A cycle is a closed path, that is, a path that begins and finishes at the same node (and no node is visited more than once).

2.3.3.3 File

In day-to-day life, all the information cannot be saved in terms of data. Sometimes a file is used to store voluminous data in an external storage device. A file is a collection of records with one or more fields. The commonly use organization for a file is:

- i. Sequential file
- ii. Relative file
- iii. Direct file
- iv. Index sequential file
- v. Index file

The basic operations that we perform with a file are the same as any other data structure.

2.4 ABSTRACT DATA TYPES (ADTS)

When it comes to solving problems with a computer, developing abstract models for the data and how the program manipulates data is a must. To sort a set of numbers, for example, we'll need an abstract model of a data structure array as well as a sorting method. To create a new model, we must first specify the abstract object we want to manage and then put the actions in place. We want to make sure that the objects are easy to use when solving a problem.

A mathematical model is an abstract data type (ADT). ADT is defined independently of the implementations. The model specifies a set of values as well as operations that can be performed on those values. A simple ADT example is a collection of numbers that can be read, sorted, searched, and printed. The ADT is a useful tool for representing the logical attributes of a data type. A data type is a collection of values and actions that are implemented with the help of a certain data structure. The basic mathematical concept that specifies the data type, pre-conditions (if any), and the operations are referred to as ADT. ADTs are generalizations of primitive data types, whereas procedures are generalizations of basic operations. The step-by-step development strategy is

- i. Mathematical model leads to informal algorithm
- ii. Abstract data type leads to algorithm
- iii. Data structures lead to program

Although most programming languages allow certain standard data types, they are insufficient for the majority of applications. Python has a derived data type known as class. However, just creating a class does not separate the operations applicable to objects of that kind. This is why the concept of ADT is useful; ADTs allow us to express whatever assumptions we have about the operations we perform on data objects.

Abstract data type is not a data structure, but it talks about the structure of the data. It tells how the data is stored and what to do with the data. These two responses describe the behavior of the data structure. A data structure is an implementation of an abstract data type.

Although the terms *data types*, *data structures*, and *abstract data type* are similar, they have distinct meanings. The data type of a variable in programming languages refers to the range of possible values for the variable. Basic types can be used to create composite data types. An abstract data type is a mathematical model that can be used to perform a variety of actions. We'll write algorithms in terms of ADTs, but to put them into practice, we'll need to figure out how to represent ADTs in terms of data types and operators. The mathematical models that underpin an ADT are expressed using data structures, which are collections of variables connected in various ways.

While establishing an abstract data type as a mathematical concept, we are not concerned with space or time efficiency. Those aren't important details.

The specification of an abstract data type includes:

1. *Instances*: The domain of values (data attributes) can be specified.
2. *Pre-conditions*: It is required when pre-conditions are mentioned.
3. *Operations*: It is a mathematical or logical process of deriving one data value from existing data values according to certain rules.

Consider the set as an ADT; the elements of the set are the domain of values, and the operations on the domain are union, intersection, complements, and so on.

The fundamental idea is that these operations are implemented once in the program, and then any other component of the program that needs to do something with the ADT may call the relevant function. If any implementation details need to be changed for whatever reason, it should be trivial to do so by simply changing the ADT operation functions. This change would go unnoticed by the rest of the software.

There is no rule defining which operations each ADT must support; this is a design decision. Some of the data structures we'll look at in this book are examples of ADT. We'll show how to implement each one, but if you do it well, the applications that use them won't need to know which one you used.

2.5 ALGORITHMS

A solution to a problem consists of the following steps:

1. Understand the problem
2. Design an algorithm
3. Convert the algorithm into a flowchart
4. Convert the flowchart into a computer program

In this section, we define an algorithm and study various methods to analyze its performance.

An algorithm is a technique used by a computer to solve a problem. An algorithm is a finite collection of instructions that performs a certain task. The following qualities must be present in all algorithms:

1. The algorithm must have zero or more inputs.
2. It must produce an output.
3. Each instruction must be clear and unambiguous.
4. The algorithm must terminate after a finite number of steps.

Algorithms are written in a language that is similar to programming. In a programming language, a program is the expression of an algorithm. There are a few distinct areas in the study of algorithms:

1. *Devise an algorithm*: For the design of an algorithm, there are various design approaches such as the greedy technique, divide and conquer technique, and dynamic programming. Some of the concepts might be used in fields other than computer science, such as operations research and electrical engineering.
2. *Validate an algorithm*: After it has been constructed, it is critical to ensure that the algorithm produces accurate results for all permissible inputs. This is referred to as algorithm validation. It is independent of the programming language in which it will be written in the future, which is the next stage in the algorithm's evolution. For the program to be legitimate, each statement must be stated, and all basic operations must be demonstrated accurately.
3. *Analyze an algorithm*: This activity, also known as performance analysis, involves deciding how much memory is required for program execution and the amount of time the central processing unit (CPU) is used. This enables us to compare one algorithm to another, both of which were created for the same goal and with the same inputs. This study also aids us in determining the algorithm's performance in the best, average, and worst cases.
4. *Test a program*: Debugging and profiling are the two steps of software testing. Debugging is the process of running a program on test data to see if the results are accurate, and if they aren't, fixing the program until it produces proper results for all legal inputs. The practice of running a program on datasets and measuring the time and space required to compute the results is known as profiling or performance measurement.

2.5.1 Algorithm specifications

2.5.1.1 Pseudo-code

A natural language like English can be used to specify an algorithm. For small and simple algorithms, graphic representations such as flowcharts are also used. We may also define algorithms in pseudo-code that resembles a computer language, making program translation simple. We may utilize conditional statements, conditional repeated statements, and finite repetitive statements while developing an algorithm using pseudo-code.

A sample algorithm:

```
Algorithm Max(a,n):
    result = a[0];
    for i in range (n):
        if(a[i] > result):
            result = a[i]
    return result
```

This method finds the largest value in an array of size n . In this case, *Max* is the name of the algorithm, and a and n are its parameters. The largest element in the array is returned as the “result” in the preceding procedure.

2.5.1.2 Recursion

If the same algorithm is invoked in the body, it is considered to be recursive. When algorithm A calls algorithm B and algorithm B calls algorithm A, it is said to be indirectly recursive.

The Towers of Hanoi problem is an example of a recursive algorithm.

Suppose we want to find the sum of the integers in an array.

```
algorithm Rsum(a,n):
    count=count+1
    if(n<=0):
        return 0
    else:
        count=count+1
        return Rsum(a,n-1) + a[n]
```

With a different set of parameters, the algorithm calls itself. The number of steps completed is shown by the count.

$$T_{\text{Rsum}}(n) = 2 + T_{\text{Rsum}}(n-1)$$

where $T_{\text{Rsum}}(n)$ indicates the total number of steps to be executed while adding the elements in an integer array of size n . In the right-hand side of the equation, the 2 indicates the number of steps executed before we call $\text{Rsum}(a, n-1)$.

The recurrence relation can be solved as follows:

$$\begin{aligned} \text{TRsum}(n) &= 2 + \text{TRsum}(n-1) \\ &= 2 + 2 + \text{TRsum}(n-2) \\ &= \dots \\ &= 2(n) + \text{TRsum}(0) = 2n + 2 \end{aligned}$$

In general, if $T(n)$ is the time taken by the algorithm when the input size is n , it can be expressed as

$$T(n) = c + T(n-1)$$

where c is some constant.

Another type of recurrence relation is

$$T(n) = a T(n/b) + f(n)$$

where the input is divided into b groups and for each group, we apply the same algorithm. $f(n)$ is the number of steps or the time taken to combine the solutions of the sub-problems. Let us take a simple example.

$$T(1) = 1$$

$$T(n) = T(n/2) + c$$

We can solve the recurrence relation as follows:

$$T(n) = T(n/2) + c$$

Substituting for $T(n/2)$,

$$\begin{aligned} T(n) &= (T(n/2^2) + c) + c \\ &= T(n/2^2) + 2c \end{aligned}$$

After k iterations

$$T(n) = T(n/2^k) + kc$$

Without any loss of generality, let us assume $n/2^k = 1$ or $k = \log_2 n$. Then

$$T(n) = T(1) + c \log_2 n = 1 + c \log_2 n$$

Therefore, $T(n) = O(\log_2 n)$.

Example

$$T(n) = 2T(n/2) + n; T(1) = 1.$$

Substituting for $T(n/2)$

$$\begin{aligned}
 &= 2[2(n/2^2) + n/2] + n \\
 &= 2 * 2T(n/2^2) + 2n
 \end{aligned}$$

After k iterations

$$= 2kT(n/2^k) + kn$$

Let us assume $n = 2^k$ or $k = \log_2 n$.

$T(n) = nT(1) + n\log_2 n$; or $T(n) = n + n \log_2 n$. Since $n\log_2 n$ dominates n as n becomes large, we write $T(n) = O(n\log_2 n)$. The big oh notation is explained later.

2.5.2 Performance analysis

The algorithm's efficiency may be assessed by measuring its performance. The performance is determined by the amount of storage space required for the program and data. This is referred to as space complexity, while the time it takes to run the program is referred to as time complexity. In this book, the focus will be more on data structures rather than performance analysis.

2.5.2.1 Space complexity

Definition: The space complexity of an algorithm is the amount of memory the algorithm needs to run until completion.

The space needed to run an algorithm consists of two parts:

1. Fixed part, which does not depend on inputs and outputs.
2. Variable part, which is the space needed by component variables.

$$S(p) = C + S_p$$

Algorithm MAX

```

{  result = a[i]
   for ( i = 2 to n)
   {  if (a[i] > result)
       result = a[i]
   }  return result
}
```

In the preceding algorithm MAX, we need one word each for n , $result$, and i , and n words for storing the array.

Therefore, $S(p) \geq n+3$ words.

Consider the following recursive part of the program.

```
Algorithm RSum(a,n)
{
    if( n<= 0) return 0.0
    else return( RSum(a, n-1) + a[n])
}
```

Since the depth of recursion is $n+1$ and each recursive call needs the value of n , $a[n]$, and sum, we need $S(p) \geq 3(n+1)$ words.

2.5.2.2 Time complexity

Definition: The time complexity $T(P)$ for a program P is the sum of compile time and execution time.

The compile time is constant and does not depend on variable characteristics (int, float, long, etc.). Therefore, $T(P) = C_p + C_a \text{ ADD}(n) + C_s \text{ SUB}(n) + \dots$, where n denotes the instance characteristic and C_a , C_s , ... represent the number of additions, subtractions, etc., that are performed on the instance characteristic n .

A programming step is a syntactically and semantically important piece of a program that executes independently of the instance characteristic. Consider the program that computes the sum of the objects in an n -dimensional array. A count variable is included in the program, which is incremented by one for each statement executed.

```
Algorithm sum(a, n)
{
    S=0.0;
    count = count+1 //count is initialized to 0.
    for i=1 to n do
    {
        count=count+1;
        S = S + a[i];
        count = count+1;
    }
    count=count+1; //last time for the for loop
    return S;
};
```

The value of count after the execution will be $2n + 3$, which indicates the number of steps executed.

Let us look at matrix addition.

```
Algorithm ADD( a,b,c,m,n)
// a, b are the matrices to be added. c is the result matrix. m>n
{
    for i=1 to m do
        for j=1 to n do
            c[i,j]=a[i,j] + b[i,j];
}
```

Let us calculate count by forming a table (see Table 2.1).

Table 2.1 Calculation of count after each statement is executed

Statement	s/e	Frequency	Total steps
Algorithm MatAdd(a,b)	0	—	0
{	0	—	0
for i=1 to m do	1	m + 1	m + 1
for j=1 to n do	1	mn + m	mn + m
c[i,j]=a[i,j]+b[i,j]	1	mn	mn
}	0	—	0
Total =		2mn + 2m + 1	2mn + 2m + 1

2.5.2.3 Complexity analysis

The complexity analysis can produce three different outcomes: worst case, best case, and average scenario. The worst-case scenario occurs when the algorithm takes a large number of steps, and the best-case scenario occurs when the number of steps is the smallest. The average situation falls midway between these two extremes. In simple cases, the average case is calculated by looking at various inputs, calculating the number of steps necessary for each one, adding the number of steps required for all inputs, and dividing by the number of inputs. If we assume that the probability of occurrence of the i_{th} input is $p(input_i)$, then we may calculate

$$T_{ave} = \sum_i p(input_i) * steps(input_i)$$

assuming that the probability distribution of the inputs is known.

In the case of a sequential search, the complexity of the three instances is rather straightforward to estimate. However, as we'll see later, determining the average case behavior for all algorithms isn't always straightforward:

Example: Find the best, worst, and average cases to search for an element sequentially in an unordered array.

Solution: When the element to be searched is the last element in the array, the worst-case scenario happens, and when the element to be searched is the first element in the array, the best-case scenario occurs.

To find the average case, let us assume that the element we are looking for can be in any one of the cells of the array. In other words, the probability of occurrence in the first cell is $\frac{1}{n}$, in the second cell is $\frac{1}{n}$, and in the n th cell is $\frac{1}{n}$. Therefore, we may find the element in one trial, two trials, or n trials.

On the average, the number of trials needed is

$$\frac{(1 + 2 + 3 + \dots + n)}{n} = \frac{n + 1}{2}$$

Example: Take binary search, for example. Assume the input size is a power of two without sacrificing generality. If the element is in the center, a binary search can find it in one try. If it's in the middle of the left or right half, it'll take two attempts, or three tries if it's in the middle of the first quarter, or the middle of the second quarter, or the middle of the fourth quarter, etc., or $\log_2 n$ tries if it's the first or last element.

Thus, the average number of possibilities is

$$1.1 + 2.2 + 4.3 + 8.4 + \dots + \frac{n}{2} \log_2 n$$

$$= \sum_0^{\log \log n - 1} 2^{(\text{power } i)} (i + 1)$$

and this has to be divided by n .

To express the complexity of an algorithm, we use Big Oh, Omega, Theta, Small Oh, and Little Omega and notations as defined and explained next.

2.5.2.3.1 Big oh

Definition: The function $f(n)$ is $O(g(n))$ if and only if there exists positive constants c and n_0 , such that $f(n) \leq cg(n)$ for all n , $n > n_0$.

If $f_1(n) \approx O(g_1(n))$ and $f_2(n) = O(g_2(n))$, then $f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n)))$ and

$$f_1(n) * f_2(n) = O((g_1(n)) * (g_2(n))).$$

Graphically, the relationship between $f(n)$ and $g(n)$ can be represented by the diagram in Figure 2.6.

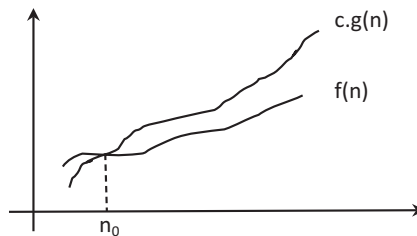


Figure 2.6 Big oh notation

Example: Let $f(n) = 4n + 3$; then $f(n) \leq 5n$ for all $n \geq 3$.

Thus, $f(n)$ is $O(n)$ with $c = 5$ and $n_0 = 3$.

Let $f(n) = 9n^2 + 5n + 3$; then $f(n) \leq 10n^2$ for $n \geq 6$.

Therefore, we say $f(n)$ is $O(n^2)$ with $c = 10$ and $n_0 = 6$.

Theorem: If $f(n) = a_m n^m + \dots + a_1 n + a_0$, then $f(n)$ is $O(n^m)$.

Proof: Since the constants $a_m, \dots, a_0$ can be positive or negative, we can write

$$f(n) \leq |a_m|n^m + \dots + |a_1|n + |a_0|$$

$$\leq n^m \sum_{i=0}^m |a_i| n^{i-m}$$

$< n^m \sum |a[m]|$ since $(i - m)$, the exponent of n in the summation is always negative and hence n^{i-m} is always less than 1. Let $c = \sum |a[m]|$. Then, $f(n) \leq cn^m$ and $f(n) = O(n^m)$.

2.5.2.3.2 Omega

Definition: The function $f(n)$ is $\Omega(g(n))$ if and only if there exist positive constants c and n_0 , such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$ (see Figure 2.7).

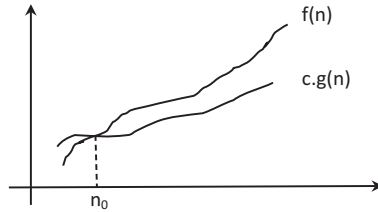


Figure 2.7 Omega notation

Example: Let $f(n) = 4n + 3$. Then $f(n) \geq 4n$ for all $n > 0$. Thus, $f(n)$ is $\Omega(n)$ with $c = 5$ and $n_0 = 3$.

Example: Let $f(n) = 6 \cdot 2^n + n^2$. Then $f(n) \geq 2^n$ for all $n \geq 1$. Hence, $f(n)$ is $\Omega(2^n)$.

We can generalize and say if $f(n) = a_m n^m + \dots + a_1 n + a_0$, then $f(n)$ is $\Omega(n^m)$.

2.5.2.3.3 Theta

Definition: The function $f(n)$ is $\Theta(g(n))$ if and only if there exist positive constants c_1, c_2 , and n_0 , such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

Example: Let $f(n) = 3n + 2$. Then $3n \leq 3n+2 \leq 4n$ for all $n \geq 2$. Hence, $f(n)$ is $\Theta(n)$.

We can generalize and say if $f(n) = a_m n^m + \dots + a_1 n + a_0$, then $f(n) = \Theta(n^m)$.

In general, exponential functions grow faster than polynomial functions and polynomial functions grow faster than logarithmic functions (Figure 2.8).

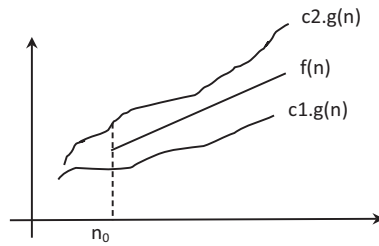


Figure 2.8 Theta notation

2.5.2.3.4 Small oh

Definition: The function $f(n)$ is $o(g(n))$ if and only if $\text{Lt } (f(n)/g(n)) = 0$ as $n \rightarrow \infty$.

Example: Let $f(n) = 3n + 2$; $g(n) = n^2$; $\text{Lt}((3n + 2)/n^2) = 0$ as n approaches ∞ . Therefore, $f(n)$ is $o(n^2)$.

2.5.2.3.5 Little omega

Definition: The function $f(n)$ is $\omega(g(n))$ if and only if $\text{Lt}(g(n)/f(n)) = 0$ as $n \rightarrow \infty$.

Example: n^2 is $\omega(n)$ since $\text{Lt}(n/n^2) = 0$ as $n \rightarrow \infty$.

True-false questions

1. Writing an algorithm is the first step when writing a program.
 - a. True
 - b. False
2. Amortized analysis is used in algorithms to determine the average performance of each operation in the worst-case scenario.
 - a. True
 - b. False
3. Big oh notation refers to the average case of time complexity.
 - a. True
 - b. False
4. Debugging and profiling are the two steps of software testing.
 - a. True
 - b. False
5. The complexity of non-primitive data structures is lower than that of primitive data structures.
 - a. True
 - b. False

Fill-in-the-blank questions

1. _____ data types are those data types that are not in a specific order.
2. Machine instructions work directly on _____ data structures.
3. The function $f(n)$ is _____ if and only if $\lim_{n \rightarrow \infty} (f(n)/g(n)) = 0$.
4. The performance is determined by the amount of _____ required for the program and _____.
5. The Towers of Hanoi problem is an example of a _____ algorithm.

Multiple-choice questions

1. Which one of the following is not true about a queue?
 - a. It is a primitive data type.
 - b. It is a non-linear data structure.
 - c. It is a hierarchical data structure.
 - d. All of the above.
2. An abstract data type (ADT) is
 - a. A small data type that cannot be instantiated.
 - b. The same as abstract class.
 - c. A data type for which only operation is defined on it can be used and nothing else.
 - d. All of the above.
3. Which of the following is true about a stack?
 - a. Data can be inserted only at one end.
 - b. Data can be inserted at both ends.
 - c. Data can be deleted from both ends.
 - d. None of the above.
4. Which one of the following is not the application of the queue data structure?
 - a. Data transfer asynchronously
 - b. Load balancing
 - c. Resource shared between various systems
 - d. Balancing of symbols
5. Traversal refers to
 - a. Putting the data in order
 - b. Reaching each element
 - c. Combining data elements
 - d. Finding a data element

6. Which of the following data structures allows deleting an element at one end and inserting it at the other end?
 - a. Stack
 - b. Queue
 - c. Array
 - d. Tree
7. Which of the following is a non-linear type data structure?
 - a. Stack
 - b. Queue
 - c. Array
 - d. None of the above
8. Which of the following is a linear type data structure?
 - a. Stack
 - b. Queue
 - c. Array
 - d. All of the above
9. Which of the following represents a hierarchical data structure?
 - a. Stack
 - b. Queue
 - c. Array
 - d. Tree
10. Stack is also called _____.
 - a. LIFO
 - b. FIFO
 - c. Tree
 - d. None of the above
11. Queue is also called _____.
 - a. LIFO
 - b. FIFO
 - c. Tree
 - d. None of the above
12. Which of the following data structures can't store non-homogeneous data elements?
 - a. Array
 - b. Stack
 - c. Queue
 - d. All of the above

-
13. Which of the following is true about the characteristics of an abstract data type?
- Has a type
 - Has an operation
 - True, False
 - False, True
 - True, True
 - False, False
14. Push and pop operation refers to
- Array
 - Stack
 - Tree
 - Graph
15. Which of the following is a hierarchical data structure?
- Array
 - Stack
 - Tree
 - Graph
16. Which of the following is very useful in situations when data have to be stored and then retrieved in reverse order?
- Array
 - Stack
 - Tree
 - Graph
17. Linear arrays are also called _____.
- Straight line array
 - Vertical array
 - Horizontal array
 - One-dimensional array
18. Which of the following is an indexed data structure?
- Tree
 - Graph
 - Stack
 - Array
19. Match the following properties of an array with their descriptions.
- | | |
|-----------------|--|
| a) Homogeneous | i) The list size is constant. |
| b) Ordered | ii) There is a first and last element. |
| c) Finite | iii) There is a next and previous in the natural order of the structure. |
| d) Fixed length | iv) Every element is the same. |

- a. a-i, b-ii, c-iii, d-iv
- b. a-ii, b-iii, c-iv, d-i
- c. a-iii, b-i, c-ii, d-iii
- d. a-iv, b-iii, c-ii, d-i

20. Which of the following is an application of stack?

- a. Infix to postfix conversion
- b. Finding factorial
- c. Tower of Hanoi
- d. All of the above

Descriptive questions

- 1. What is a data structure?
- 2. What is the difference between primitive data and non-primitive data?
- 3. Explain the different operations on a data structure.
- 4. Write briefly about different data structures.
- 5. Explain the terms data type, data structures, and abstract data type.

Answers to true-false questions

- 1. False
- 2. True
- 3. False
- 4. True
- 5. False

Answers to fill-in-the-blank questions

- 1. Non-linear
- 2. primitive
- 3. $O(g(n))$
- 4. storage space, data
- 5. recursive

Answers to multiple-choice questions

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. d | 2. c | 3. a | 4. d | 5. b |
| 6. b | 7. d | 8. d | 9. d | 10. a |
| 11. b | 12. d | 13. c | 14. b | 15. c |
| 16. b | 17. d | 18. d | 19. d | 20. d |